Software Engineering Practice Quiz 2

Name: Wang Hao (汪浩)
NUIST ID: 202383890022 SETU ID: 20109992

1 What is Object-Oriented Analysis and Design?

Object-oriented analysis identifies and describes conceptual objects in the problem domain and their relationships. Object-oriented design defines software objects, responsibilities, interfaces and collaborations that implement the requirements. For a POS, analysis identifies a Sale and its lines; design decides that Sale calculates its total by asking line objects for their subtotals. The conceptual and software models are related but not identical.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 1.

2 What are three Ways to Apply UML?

UML as sketch uses informal, selective diagrams for exploring and communicating ideas. UML as blueprint uses more detailed diagrams as specifications for implementation or documentation. UML as programming language uses sufficiently complete executable models from which behavior or code is generated. The level of detail should serve the project rather than become an end in itself.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 1.

3 What are Benefits to Iterative Development?

Iterations expose technical and requirement risks early, provide frequent stakeholder feedback, and allow priorities to change as understanding improves. Integrated working increments make progress visible and test assumptions. Teams can improve their process and design between cycles, while continuous testing limits the accumulation of integration defects. Benefits require real increments rather than repeating analysis-only phases.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 2.

4 Why is the Waterfall so Failure-Prone?

A rigid waterfall assumes requirements can be stabilized and design completed before implementation provides meaningful feedback. In complex systems, important needs and risks often emerge during use or coding. Late integration and testing then make misunderstandings costly to correct. The weakness is delaying learning while treating early assumptions as settled. Stable, well-understood tasks may still use sequential steps appropriately.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 2.

5 How to do Iterative and Evolutionary Analysis and Design?

Begin with a broad vision, key actors and a high-level list of goals. Select high-value or risky scenarios for the next short iteration. Analyze those scenarios in enough detail, sketch the necessary domain and design models, implement and test a working increment, then obtain feedback. Refine requirements and design for the next cycle rather than attempting a complete detailed specification first.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 2, 6, 9 and 18.

6 How to Find Use Cases?

Define the system boundary, identify primary actors and ask which goals they want the system to help achieve. List actor-goal pairs and turn useful user goals into use cases named with verb phrases. Check that each delivers observable business value at a suitable level, using the boss test, elementary-business-process test and size test. Write brief cases first, then elaborate selected important ones.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 6.

7 How to Find Conceptual Classes for a domain model?

Examine requirements and use-case narratives for domain vocabulary, consult domain experts and reuse relevant domain models. Use conceptual-class categories such as transactions, line items, descriptions, roles, places and events as prompts. Noun phrases are candidates, not an automatic answer. Retain concepts with meaningful domain identity or information, distinguish classes from simple attributes, and avoid software artifacts such as controllers in the analysis model.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 9.

8 When Are Description Classes Useful in creating a domain model?

A description class is useful when information describes a kind of thing and must exist independently of individual instances, when repeated description data should be shared, or when deleting instances must not remove the description. ProductDescription can retain an item identifier, name and current catalog price even when no physical item is in stock. A sale line separately retains the historical transaction price.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 9.

9 What are System Sequence Diagrams? Why Draw an SSD?

An SSD shows a scenario as ordered events between external actors and the system treated as one black box, together with relevant responses. It identifies system operations at the boundary without deciding internal software collaborations. Drawing an SSD clarifies the events needed to support a use case and provides input for operation contracts and object design.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 10.

10 What is Software Architecture?

Software architecture describes the important structures and organization of a system, its major elements and their relationships, and significant decisions and constraints that guide its development. It addresses qualities such as modifiability, performance and reliability. Several views may be needed, including logical structure, deployment and process behavior. A class diagram alone does not capture all architectural concerns.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 13 and 33.

11 What are Design Patterns? What are benefits of Design Patterns?

Design patterns describe named, reusable approaches to recurring design problems in a context, including responsibilities, relationships and consequences. They are adaptable design knowledge rather than finished code. Patterns help communicate decisions, reuse experience and reason about tradeoffs such as coupling and flexibility. Applying a pattern without the corresponding problem can add unnecessary complexity.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 17, 25 and 26.

12 What are four common ways that visibility can be achieved from object A to object B in OO design?

Attribute visibility: A holds a reference to B in an attribute. Parameter visibility: A receives B as a method argument. Local visibility: A creates B locally or obtains it as a local result. Global visibility: B is accessible through a global reference. Visibility enables A to send B a message. Global access can introduce hidden coupling and should be used cautiously.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 19.

13 Please give the definition of the Information Expert pattern.

Assign a responsibility to the class that has the information required to fulfill it. If the information is distributed, collaborating experts can each perform their part. SalesLineItem knows quantity and original price, so it calculates subtotal. Sale knows its lines, so it calculates the total by summing line subtotals. Consider coupling and cohesion when deciding how far to apply the guideline.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 17.

14 In OO design, what is a Controller? What is the basic functionality of a controller?

A GRASP Controller is an object outside the UI layer that receives and coordinates a system operation. It may represent the overall system or a use-case/session handler. It accepts requests, obtains or creates appropriate objects and delegates domain work. It should not absorb all business logic. A controller is responsible for coordinating system events, not for drawing screens.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 17.

15 What is the Adapter pattern for OO design? When to use Adapter pattern?

Adapter converts or wraps an existing component interface into the interface a client expects, allowing otherwise incompatible components to collaborate. Use it to isolate a third-party API, integrate a legacy component, or replace a service while protecting clients from interface changes. A POS payment adapter could translate a uniform authorize request into a provider-specific protocol. It does not itself define the payment business policy.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 26.